AES allows keys to be in one of three different sizes: 128-bit, 192-bit, or 256-bit. In principle, correctly guessing a 128-bit key is well beyond the capability of even an attacker with enormous amounts of computing power. Trying every possible value of a key is known as a *brute-force attack* and should be impossible for a key of that size, and this is generally assumed to be true for a single key. There are three general ways in which that assumption might prove to be wrong:

- A weakness in the encryption algorithm might be discovered that reduces the amount of effort required to crack the key. Increasing the size of the key provides a security margin against such a possibility.
- New types of computers might be developed that can perform brute force search much quicker than existing computers. This is believed to be true of *quantum computers*, but it's not known whether it will ever be possible to build a large enough quantum computer for this to be a real threat. Doubling the size of the key protects against known quantum attacks for symmetric algorithms like AES.
- Theoretically, if each user has their own encryption key and you have millions of users, it may be possible to attack every key simultaneously for less effort than you would expect from naively trying to break them one at a time. This is known as a *batch attack* and is described further in https://blog.cr.yp.to/20151120-batchattacks.html.

At the time of writing, none of these attacks are practical for AES, and for short-lived authentication tokens the risk is significantly less, so 128-bit keys are perfectly safe. On the other hand, modern CPUs have special instructions for AES encryption so there's very little extra cost for 256-bit keys if you want to eliminate any doubt.

Remember that the JWE CBC with HMAC methods take a key that is twice the size as normal. For example, the `A128CBC-HS256` method requires a 256-bit key but this is really two 128-bit keys joined together rather than a true 256-bit key.

6.3.4 Using a JWT library

Due to the relative complexity of producing and consuming encrypted JWTs compared to HMAC, you'll continue using the Nimbus JWT library in this section. Encrypting a JWT with Nimbus requires a few steps, as shown in listing 6.6.

- First you build a JWT claims set using the convenient `JWTClaimsSet.Builder` class.